

4. MITM- monkey patching

Preconditions

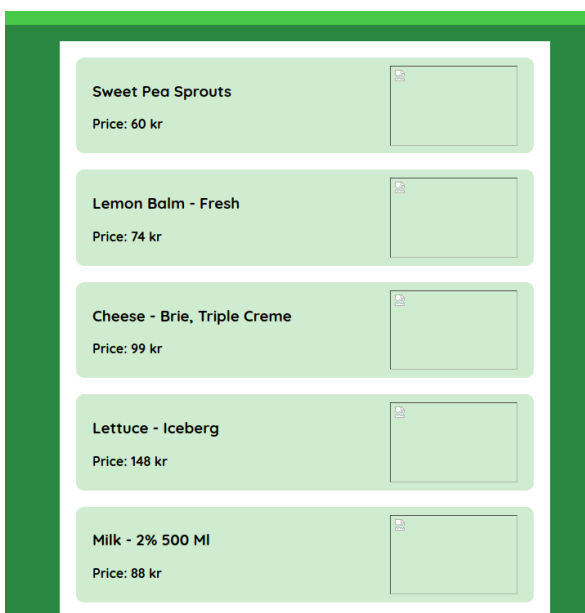
The file has been downloaded from the nodehill website, processed through the Visual Code and initiated with node index.js on localhost:3005. The website runs well until certain point but then shuts down for unknown reasons, which kills operations mid-way.

Challenge 1

Get the site to show the price at 20% discount without peeking in the source code after the discount code

Sequence of events

At the start the website displays a field asking to enter a discount code. Before engaging in the attack, start with a reconnaissance. I enter a code (any number will do at this point) and the website proceeds to list a bunch of products with prices (screenshot)



Upon refreshing the website → Console →

```
let oldFetch = fetch;
```

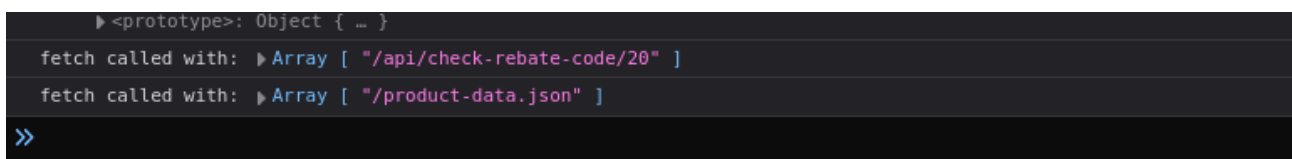
```
fetch = function (...args) {  
  console.log("fetch called with:", args);  
  return oldFetch(...args);  
};
```

→ enter discount value again

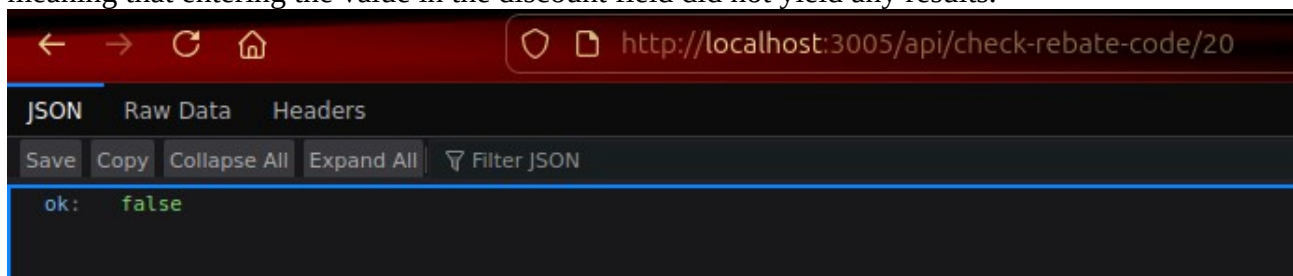
→ values returned in the console

```
fetch called with: Array [ "/api/check-rebate-code/20" ] debugger eval code:4:11
```

```
fetch called with: Array [ "/product-data.json" ]
```



The *check-rebate-code* API invalidates current discount attempt of arbitrary value (*ok: false*), meaning that entering the value in the discount field did not yield any results.



In order to inject desired discount → Refresh the website → in Console run the code (explanation provided line by line):

`let oldFetch = fetch;` → backups original fetch function so the app still runs flawlessly

`fetch = async function (...args) {` → replaces browser's fetch function with a custom function of asynchronous nature that's similar in mechanics to fetch (meaning that it works on promises). `Args` collects arguments passed to this function. This is where the MITM part of the attack starts.

`let [url] = args;` → extracts the url from the arguments passed into the above function

`let response = await oldFetch(...args);` → sends the response to the real server, based on the original fetch function as a real request. Saves it in the `response` object.

`let oldJson = response.json;` → stores the original json method so it can be called later inside the modified version.

`response.json = async function () {` → the core of the attack. The data in `new json` will be called into the attacker's function.

`let data = await oldJson.apply(response);` → calls the real json method into a js object

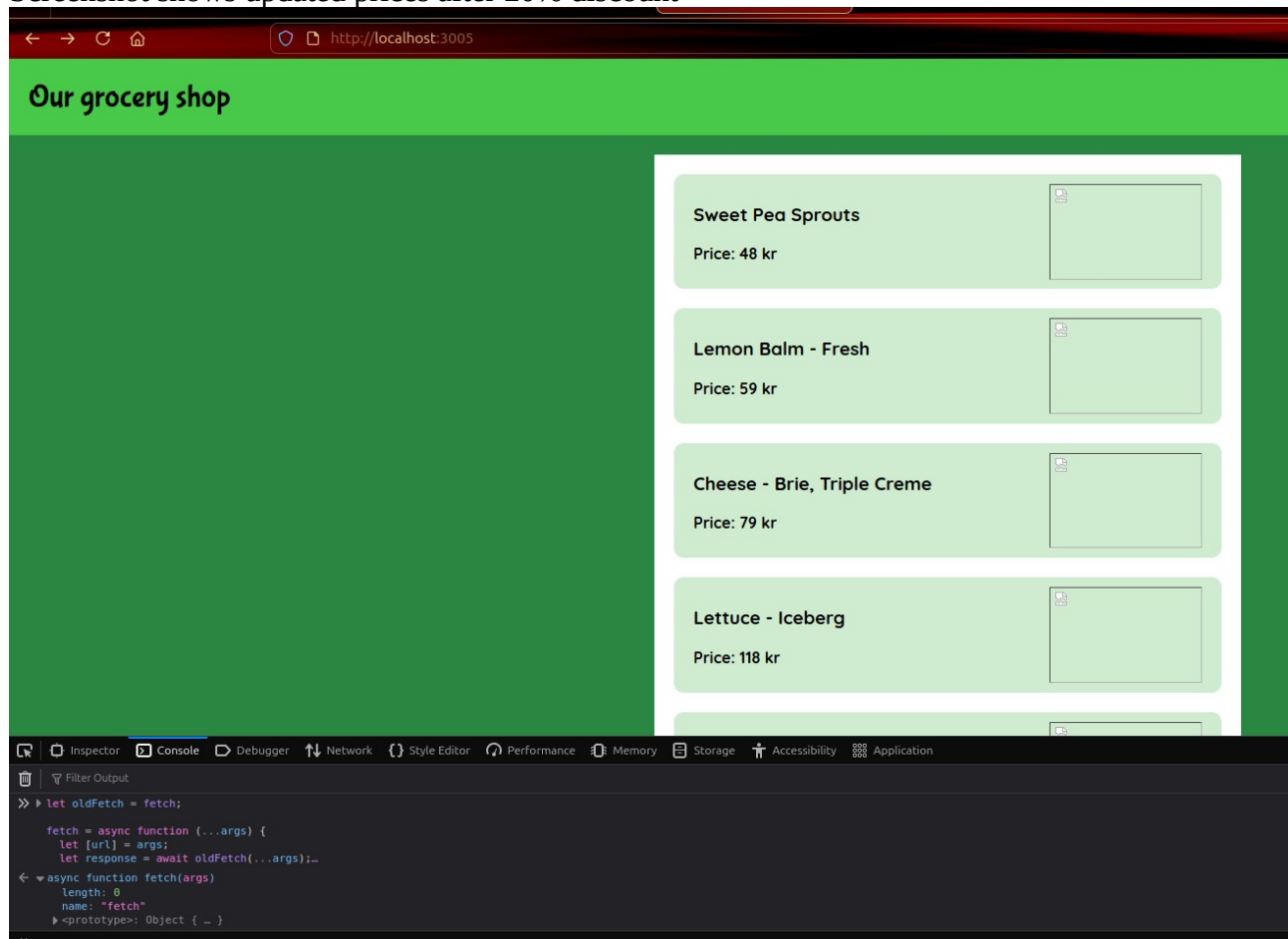
`if (url.startsWith('/api/check-rebate-code/')) {` → checks if the request validates the discount

`data = { ok: true };}` → changes core security principle: makes the website believe that whatever value goes through the discount field is real

`return data;}` → returns modified json to the frontend

`return response;}` → returns attacker's function with monkey-patched json allowing the application to continue.

Screenshot shows updated prices after 20% discount



Why did the attack work? What was it in the code that made it possible?

The attack worked because the frontend accepted the result of `response.json()` from `fetch('/api/check-rebate-code/...')` without server-side enforcement, allowing the fetch function to be overridden so `{ ok: true }` could be returned regardless of the real server response.

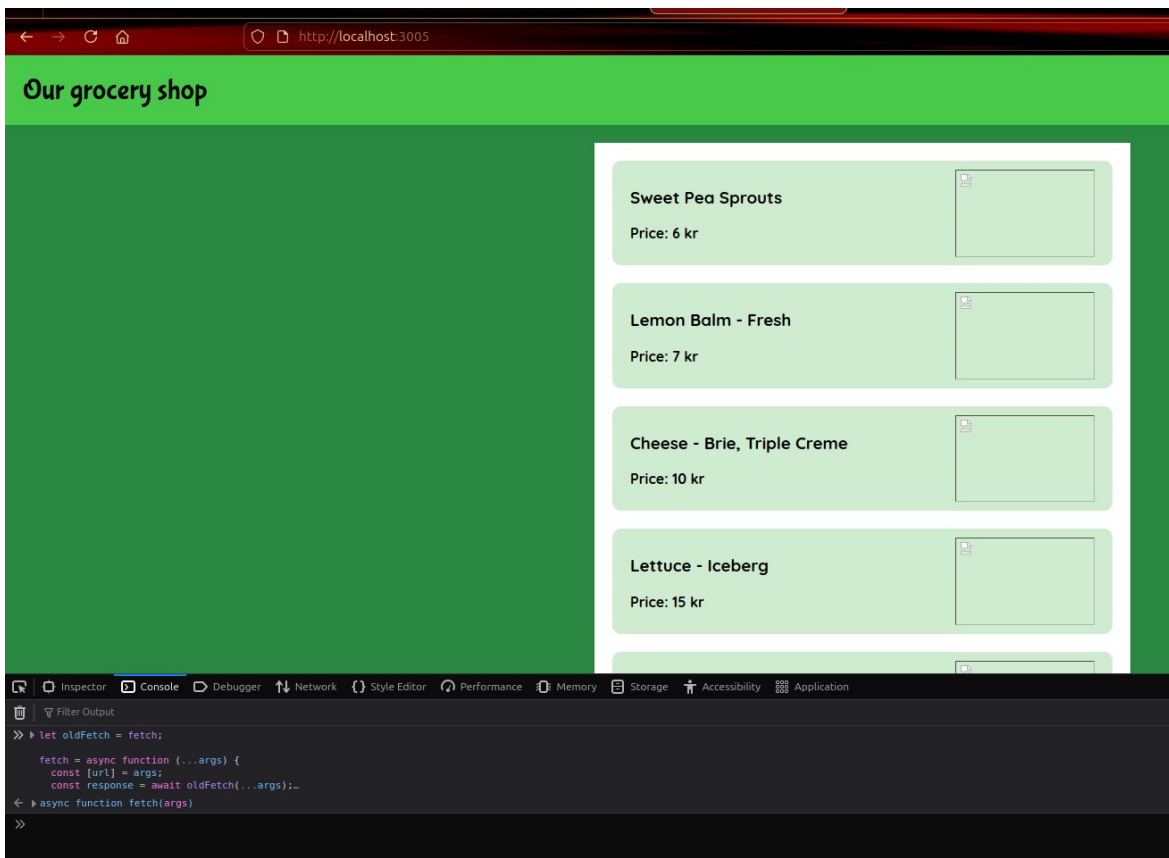
Challenge 2

Put any price on all products, e.g. a tenth of the price?

From the base line the new code will put in the console. Restart website → Console → enter the code. Only the explanation to new pieces of code provided.

```
let oldFetch = fetch;
fetch = async function (...args) {
  const [url] = args;
  const response = await oldFetch(...args);
  const oldJson = response.json;
  response.json = async function () {
    const data = await oldJson.apply(response); → creates a json method with data as an array of
    products that's parsed into js value.
    if (url === '/product-data.json') { → checks if the current response has product data, only this
    API is modified.
      for (const product of data) { → this loop goes through every product object in the array...
        product.price *= 0.1; } } ... and multiplies each of them by 0.1 reducing the price 90%.
    return data; };
  return response;};
```

Screenshot demonstrates new prices after implementation



Why did the attack work? What was it in the code that made it possible?

The attack worked because the frontend trusted json returned by monkey patched `/product-data.json` API so when the page called `response.json()`, the intercepted the array of products was modified changing each product's price individually by arbitrary value. In the case of challenge two

the value entered in the discount field didn't matter and the vulnerability bypassed direct user interface.

Which category in the OWASP Top 10 belongs to the vulnerability?

Both challenge 1 and 2 present evidence of **OWASP A06:2025 Insecure Design** and **OWASP A05:2025 Injection** (to lesser extent)

The attack illustrates that the website trusts client-side input while the security logic must be enforced server-side to prevent code manipulation.

1. Is it a security risk that you can fool the site to show discounted prices?

Yes, quite obviously it is a security risk resulting in financial loss of a company and it doesn't even require a significant system knowledge to perform. In the episode of 56 ("Jordan") of Darknet Diaries a similar situation is presented when a 13-year-old teenager fooled pizza stores and various types of businesses to grant himself discounts on consumer products. These were early 90s and the fact that these attacks are superficial in their nature and still present 3 decades later is a threat on its own. The attacker doesn't need to break into servers or networks but exploits critical logic of the client-side trust.

The core vulnerability presented in those challenges lies in the fundamental flaw where frontend allows user input and treats it as a trusted value. Browsers are powerful tools, any user with access to the console can manipulate requests, responses, and code behaviour. In the current example the attacker can easily manipulate MITM attack in their own environment altering business logic/strategy by changing the code. From security perspective this can be considered Broken Access Control and Insecure Direct Object References (IDOR), because the server implicitly trusts client-entered data without verification.

This points to a pattern in web security architecture where separation from presentation logic and security-critical logic is missing. The security of the website is only as strong as its backend and those critical designs should occur server-side. Any data returned to the client should be considered read-only and untrusted. Even sophisticated measures like JWT tokens or signed payloads can be circumvented if the server does not validate each request. Otherwise there is a risk of creating an easily accessible attack surface where both fiscal and reputational aspects of a business can be effortlessly bent.

Finally, the above examples show that any client-facing code is vulnerable to attacker's control. This extends to but is not limited to mobile clients and third party implementations. Therefore the best way may be to shift the trust model entirely to backend and using the client only as a presentation layer.

2. Could it have been avoided?

Yes, this vulnerability could have been avoided if the the frontend was treated as untrusted and laid out all critical security design onto the server. By calculating final prices and validating discount codes exclusively server-side, the system would be resilient against any client-side manipulation, like monkey-patching fetch or injecting input via console. The root cause is in a design flaw which is trusting the client indiscriminately. As the example shows, even minimal attacks can exploit this flaw, so an architecture logic that separates presentation from security by ensuring the server always has the final say over final price calculations and fiscal validation is the best way forward.

Can you prevent an order from actually being placed with the lower prices if there had been a shopping cart and checkout?

Yes, as mentioned above, this can be prevented by ensuring that all price calculations and validations are happening on the server-side during checkout, independently of what the client enters via his browser. Even if an attacker manipulates prices in the browser or shopping cart, the server must recompute the final price based on product IDs and pricing before accepting payment or creating an order. By establishing a baseline that all client-submitted prices are not to be trusted and forcing server-side validation at checkout, any client-side MITM or monkey-patching attack becomes ineffective and the order will be placed at the correct price.

Risk assessment

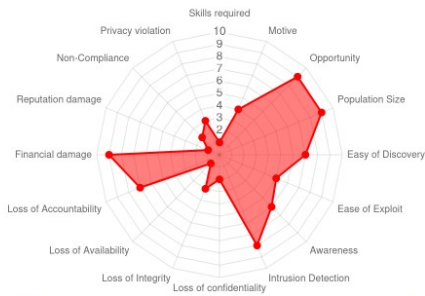
MITM attack with monkey patching rates as high on the OWASP vulnerability score, because it requires very little technical skill and resources. It can be executed entirely from the browser console. Anyone can inspect network requests and override JavaScript functions such as *fetch*, effectively resulting in a MITM attack. Since the dev tools are naturally built-in in most browsers the barrier to corruption is extremely low. The attack itself doesn't require special access, malware planting, or network control.

The caveat is however, that the impact depends solely on the backend architecture. In the workshop setup the attack altered prices because the frontend trusted the returned data and used it directly in the interface. If a real production system also trusted those client-set prices during checkout, the impact could potentially lead to bankruptcy, allowing people to purchase goods at grotesquely reduced prices.

In real life, a well-designed e-commerce system should implement in-depth server defense. Even if the frontend is manipulated, the backend should recompute prices, validate discount codes, and verify totals during checkout. If these checks exist, the attack would only affect the display, not the final transaction.

Overall, the risk severity ranges from medium to high depending on backend design. If the server relies on client-provided prices, the risk is high due to the combination of publicly accessible elementary hacking methods and direct financial impact on the company. If proper server-side validation is implemented, the same hack becomes mostly cosmetic and the risk drops immediately.

OWASP Risk Assessment Calculator



HIGH

Threat Agent Factors

Skill level	No technical skills (1)	▼
Motive	Possible reward (4)	▼
Opportunity	No access or resources required (9)	▼
Size	Anonymous Internet users (9)	▼

Technical Impact Factors

Loss of confidentiality	Minimal non-sensitive data disclosed (2)	▼
Loss of integrity	Minimal seriously corrupt data (3)	▼
Loss of availability	Minimal secondary services interrupted (1)	▼
Loss of accountability	Possibly traceable (7)	▼

Vulnerability Factors

Ease of discovery	Easy (7)	▼
Ease of exploit	Easy (5)	▼
Awareness	Obvious (6)	▼
Intrusion detection	Logged without review (8)	▼

Business Impact Factors

Financial damage	Bankruptcy (9)	▼
Reputation damage	Minimal damage (1)	▼
Non-compliance	Minor violation (2)	▼
Privacy violation	One individual (3)	▼

Likelihood score

6.125 HIGH

Impact score

3.500 MEDIUM